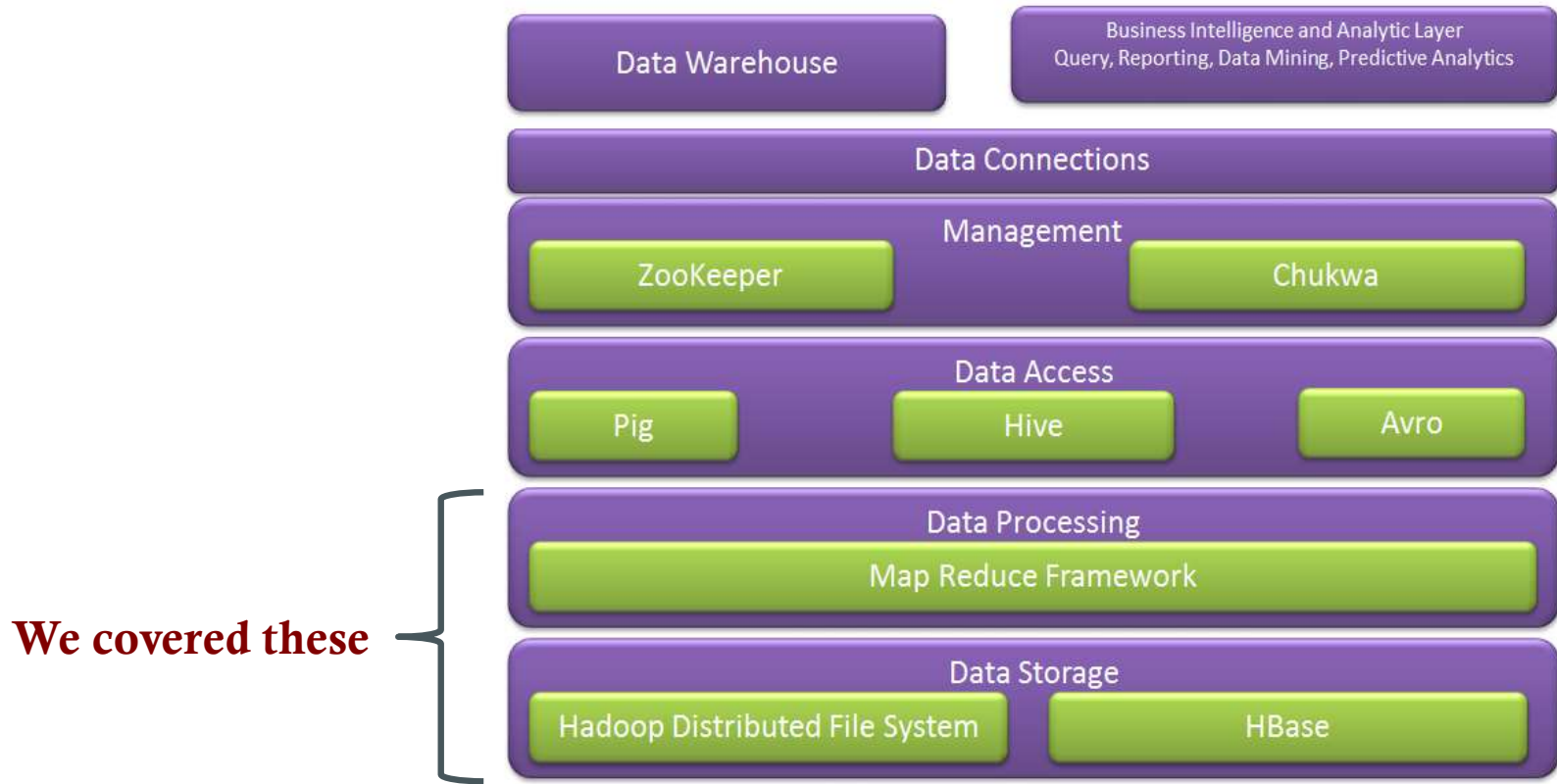


PIG and HIVE

Large-Scale Data Management

Hadoop Ecosystem



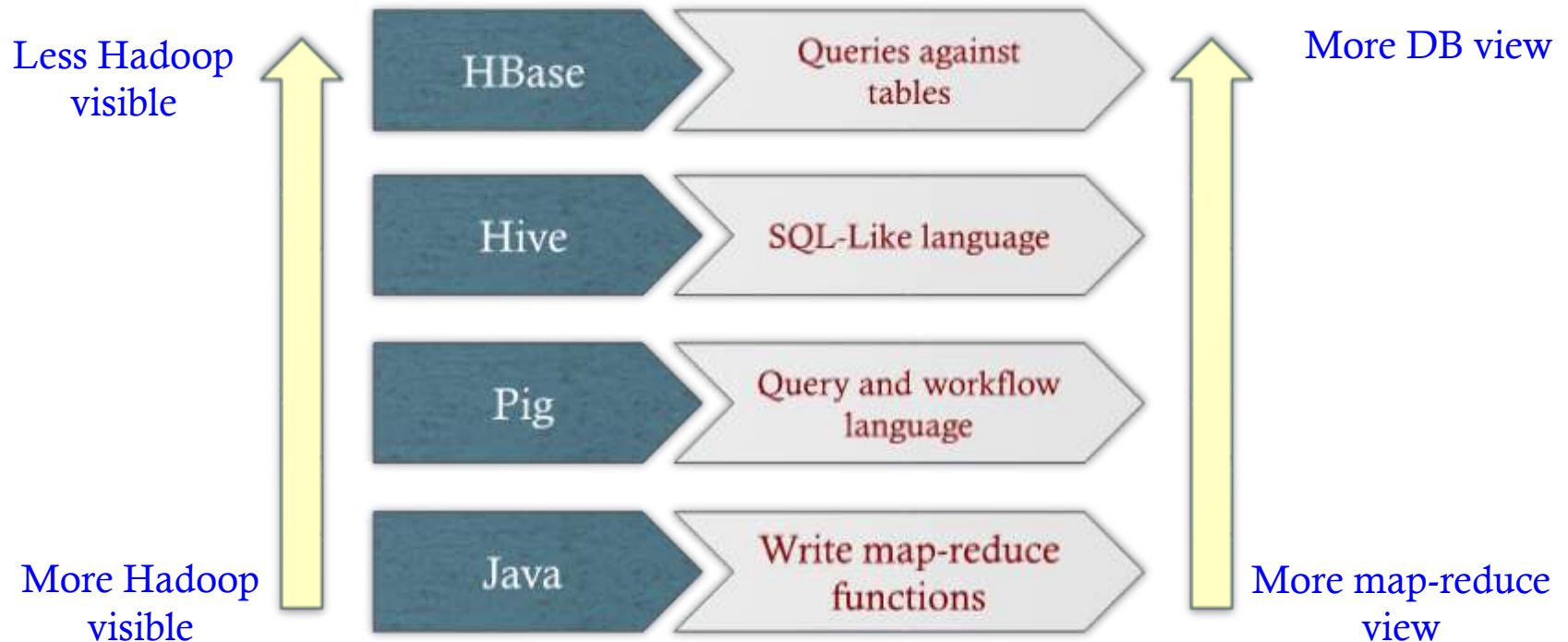
Query Languages for Hadoop

- **Java:** Hadoop's Native Language
- **Pig:** Query and Workflow Language
- **Hive:** SQL-Based Language
- **HBase:** Column-oriented Database for MapReduce

Java is Hadoop's Native Language

- Hadoop itself is written in Java
- **Provided Java APIs**
 - For mappers, reducers, combiners, partitioners
 - Input and output formats
- Other languages, e.g., Pig or Hive, convert their queries to Java MapReduce code

Levels of Abstraction



Java Example

map

reduce

Job conf.

```
File Edit Options Buffers Tools Java Help

public class WordCount {

    public static class Map extends MapReduceBase implements
        Mapper<LongWritable, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable>
            output, Reporter reporter) throws IOException {
            String line = value.toString();
            StringTokenizer tokenizer = new StringTokenizer(line);
            while (tokenizer.hasMoreTokens()) {
                word.set(tokenizer.nextToken());
                output.collect(word, one);
            }
        }
    }

    public static class Reduce extends MapReduceBase implements
        Reducer<Text, IntWritable, Text, IntWritable> {
        public void reduce(Text key, Iterator<IntWritable> values, OutputCollector<Text,
            IntWritable> output, Reporter reporter) throws IOException {

            int sum = 0;
            while (values.hasNext()) { sum += values.next().get(); }
            output.collect(key, new IntWritable(sum));
        }
    }

    public static void main(String[] args) throws Exception {
        JobConf conf = new JobConf(WordCount.class);
        conf.setJobName("wordcount");
        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);
        conf.setMapperClass(Map.class);
        conf.setCombinerClass(Reduce.class);
        conf.setReducerClass(Reduce.class);
        conf.setInputFormat(TextInputFormat.class);
        conf.setOutputFormat(TextOutputFormat.class);
        FileInputFormat.setInputPaths(conf, new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));

        JobClient.runJob(conf);
    }
}

---:--- mapreduce.java All L9 (Java/l Abbrev)-----
Wrote /home/shivnath/Desktop/mapreduce.java
```

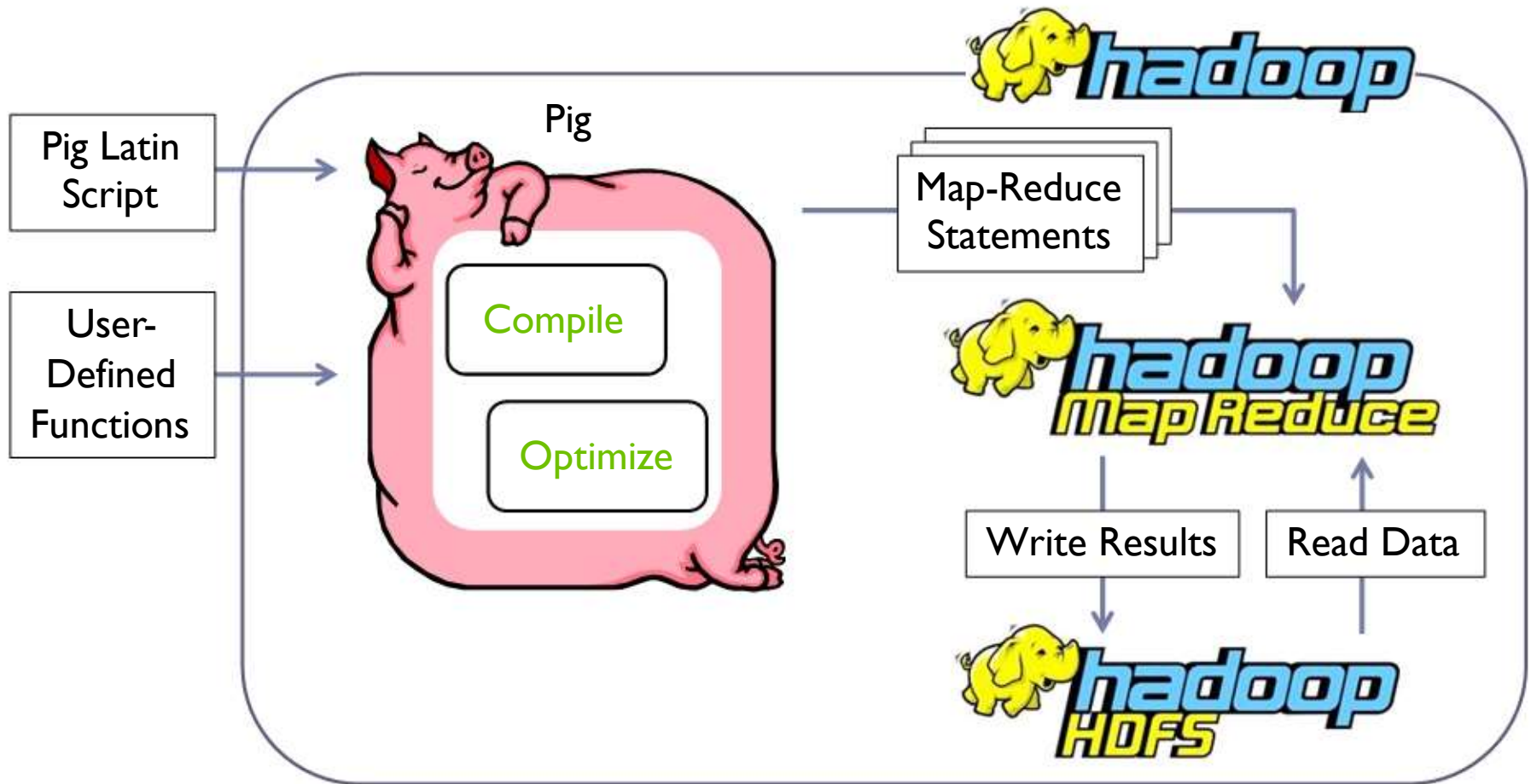
Apache Pig

What is Pig

- A platform for analyzing large data sets that consists of a high-level language for **expressing** data analysis programs.
- Apache Pig is an abstraction over MapReduce. It is a tool/platform which is used to analyze larger sets of data representing them as data flows. Pig is generally used with **Hadoop**; we can perform all the data manipulation operations in Hadoop using Apache Pig.
- To write data analysis programs, Pig provides a high-level language known as **Pig Latin**. This language provides various operators using which programmers can develop their own functions for reading, writing, and processing data.
- Compiles down to MapReduce jobs
- Developed by Yahoo!
- Open-source language



Big Picture



Why Do We Need Apache Pig?

Using Pig Latin, programmers can perform MapReduce tasks easily without having to type complex codes in Java.

Apache Pig uses multi-query approach, thereby reducing the length of codes. For example, an operation that would require you to type 200 lines of code (LoC) in Java can be easily done by typing as less as just 10 LoC in Apache Pig. Ultimately Apache Pig reduces the development time by almost 16 times.

Pig Latin is SQL-like language and it is easy to learn Apache Pig when you are familiar with SQL.

Apache Pig provides many built-in operators to support data operations like joins, filters, ordering, etc. In addition, it also provides nested data types like tuples, bags, and maps that are missing from MapReduce.

Features of Pig

Apache Pig comes with the following features –

- **Rich set of operators** – It provides many operators to perform operations like join, sort, filter, etc.
- **Ease of programming** – Pig Latin is similar to SQL and it is easy to write a Pig script if you are good at SQL.
- **Optimization opportunities** – The tasks in Apache Pig optimize their execution automatically, so the programmers need to focus only on semantics of the language.
- **Extensibility** – Using the existing operators, users can develop their own functions to read, process, and write data.
- **UDF's** – Pig provides the facility to create **User-defined Functions** in other programming languages such as Java and invoke or embed them in Pig Scripts.
- **Handles all kinds of data** – Apache Pig analyzes all kinds of data, both structured as well as unstructured. It stores the results in HDFS.

Apache Pig Vs MapReduce

Listed below are the major differences between Apache Pig and MapReduce.

Apache Pig	MapReduce
Apache Pig is a data flow language.	MapReduce is a data processing paradigm.
It is a high level language.	MapReduce is low level compared to pig and rigid.
Performing a Join operation in Apache Pig is pretty simple.	It is quite difficult in MapReduce to perform a Join operation between datasets.
Any novice programmer with a basic knowledge of SQL can work conveniently with Apache Pig.	Exposure to Java is must to work with MapReduce.
Apache Pig uses multi-query approach, thereby reducing the length of the codes to a great extent.	MapReduce will require almost 20 times more the number of lines to perform the same task.
There is no need for compilation. On execution, every Apache Pig operator is converted internally into a MapReduce job.	MapReduce jobs have a long compilation process.

Apache Pig Vs SQL

Listed below are the major differences between Apache Pig and SQL.

In addition to below differences, Apache Pig Latin –

- Allows splits in the pipeline.
- Allows developers to store data anywhere in the pipeline.
- Declares execution plans.
- Provides operators to perform ETL (Extract, Transform, and Load) functions.

Pig	SQL
Pig Latin is a procedural language.	SQL is a declarative language.
In Apache Pig, schema is optional. We can store data without designing a schema (values are stored as \$01, \$02 etc.)	Schema is mandatory in SQL.
The data model in Apache Pig is nested relational .	The data model used in SQL is flat relational .
Apache Pig provides limited opportunity for Query optimization .	There is more opportunity for query optimization in SQL.

Apache Pig Vs Hive

Both Apache Pig and Hive are used to create MapReduce jobs. And in some cases, Hive operates on HDFS in a similar way Apache Pig does. In the following table, a list a few significant points that set Apache Pig apart from Hive is mentioned.

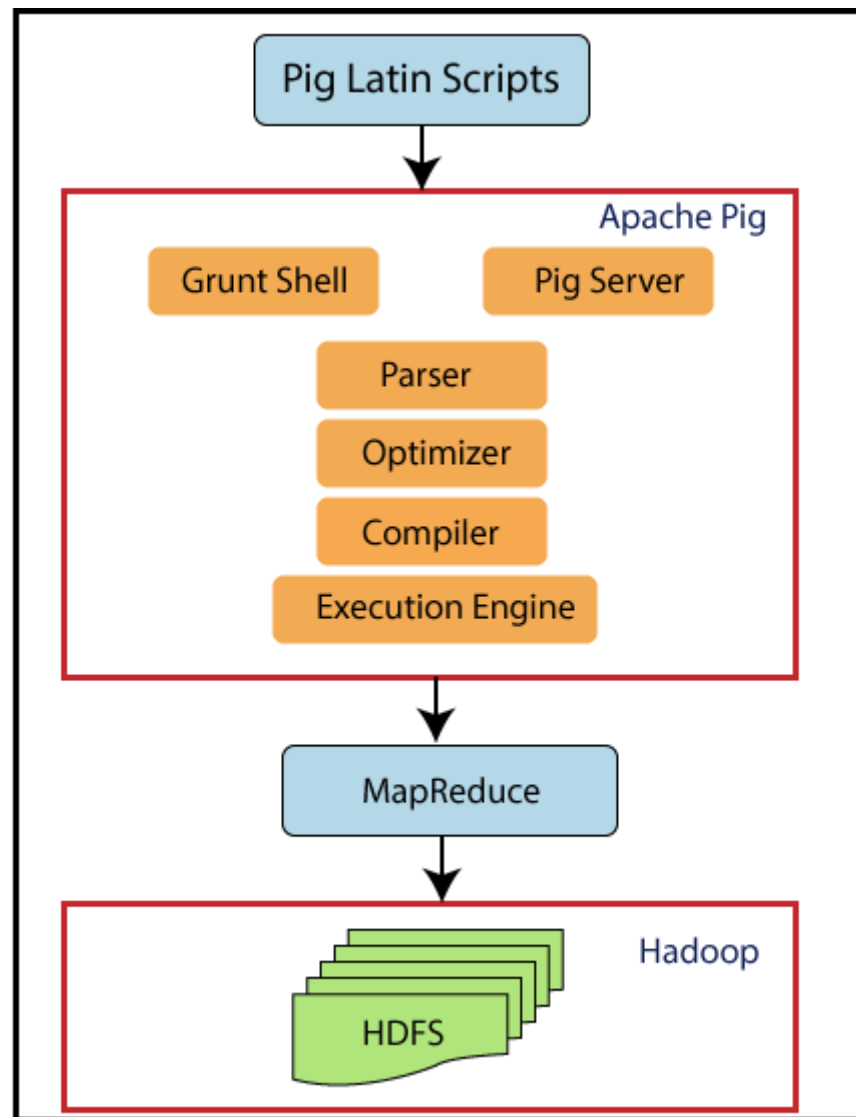
Apache Pig	Hive
Apache Pig uses a language called Pig Latin . It was originally created at Yahoo .	Hive uses a language called HiveQL . It was originally created at Facebook .
Pig Latin is a data flow language.	HiveQL is a query processing language.
Pig Latin is a procedural language and it fits in pipeline paradigm.	HiveQL is a declarative language.
Apache Pig can handle structured, unstructured (upto some extent), and semi-structured data.	Hive is mostly for structured data.

Applications of Apache Pig

Apache Pig is generally used by data scientists for performing tasks involving ad-hoc processing and quick prototyping. Apache Pig is used –

- To process huge data sources such as web logs.
- To perform data processing for search platforms.
- To process time sensitive data loads.

Apache Pig - Architecture



Apache Pig Components

Parser

Initially the Pig Scripts are handled by the Parser. It checks the syntax of the script, does type checking, and other miscellaneous checks. The output of the parser will be a DAG (directed acyclic graph), which represents the Pig Latin statements and logical operators.

Optimizer

The logical plan (DAG) is passed to the logical optimizer, which carries out the logical optimizations such as projection and pushdown.

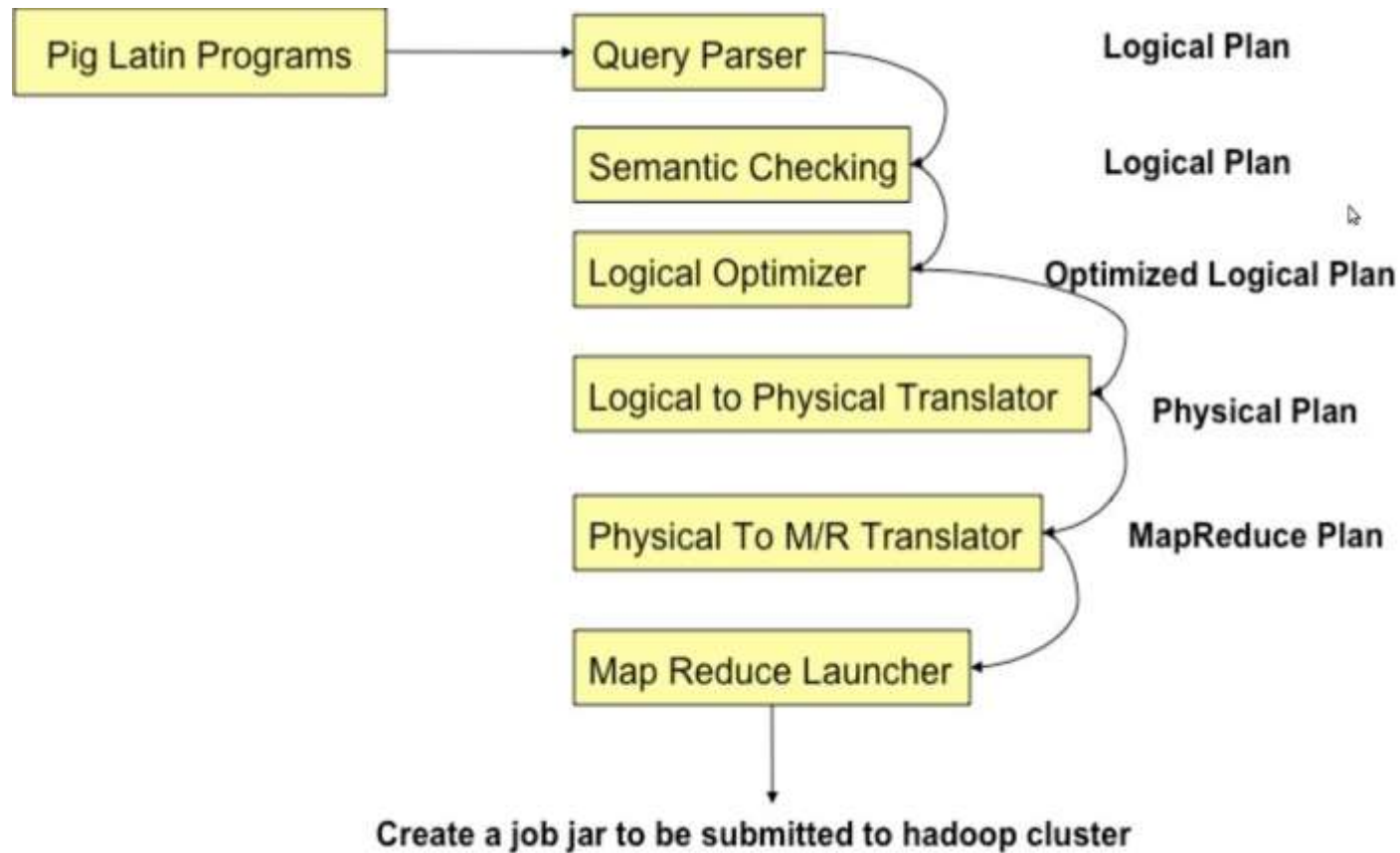
Compiler

The compiler compiles the optimized logical plan into a series of MapReduce jobs.

Execution engine

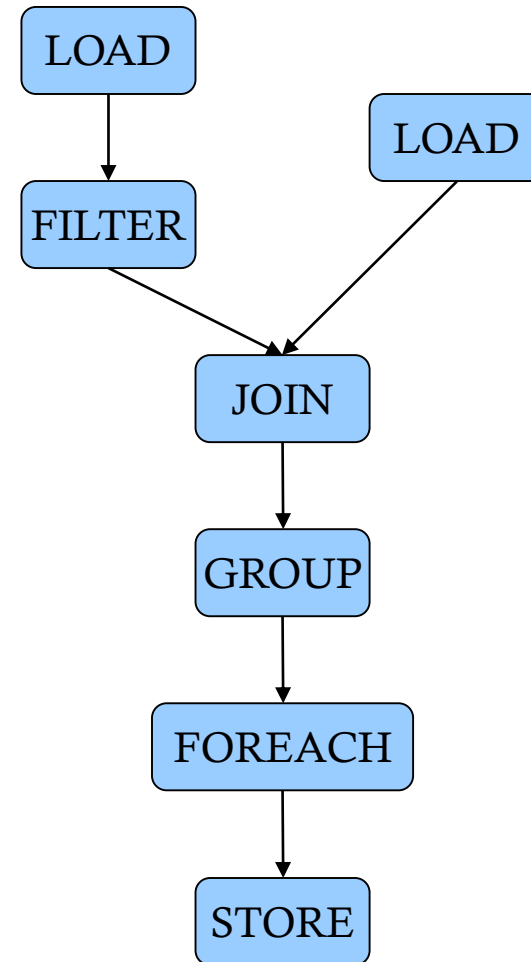
Finally the MapReduce jobs are submitted to Hadoop in a sorted order. Finally, these MapReduce jobs are executed on Hadoop producing the desired results.

Pig Compilation



Logic Plan

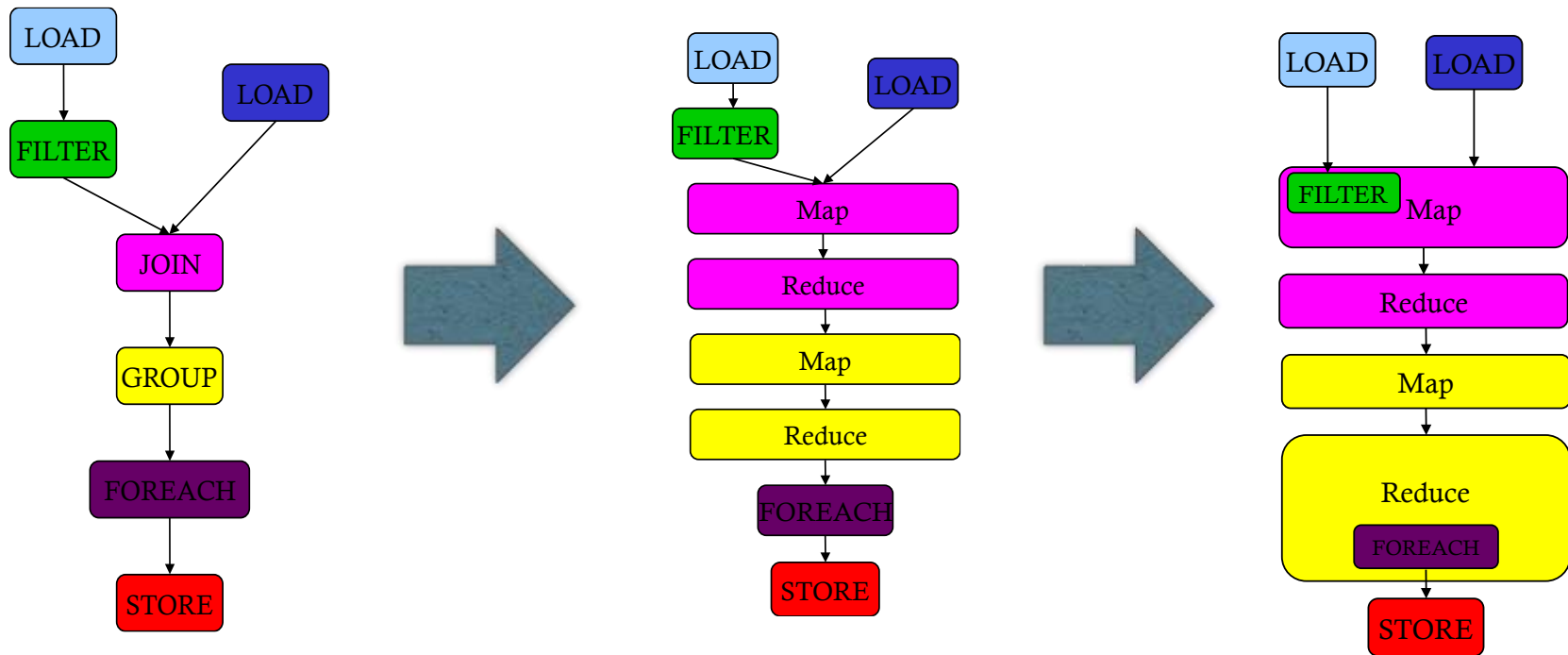
```
A=LOAD 'file1' AS (x, y, z);  
B=LOAD 'file2' AS (t, u, v);  
C=FILTER A by y > 0;  
D=JOIN C BY x, B BY u;  
E=GROUP D BY z;  
F=FOREACH E GENERATE  
    group, COUNT(D);  
STORE F INTO 'output';
```



Physical Plan

- 1:1 correspondence with the logical plan
- **Except for:**
 - Join, Distinct, (Co)Group, Order
- Several optimizations are done automatically

Generation of Physical Plans



If the Join and Group By are on the same key → The two map-reduce jobs would be merged into one.

Data Model

- “ Atom - simple atomic value (ie: number or string)
- “ Tuple
- “ Bag
- “ Map

$$\left(\text{'alice'}, \left\{ \begin{array}{l} \text{'lakers', 1} \\ \text{'iPod', 2} \end{array} \right\}, [\text{'age'} \rightarrow 20] \right)$$

Data Model

- “ Atom
- “ Tuple - sequence of fields; each field any type
- “ Bag
- “ Map

$$\left(\text{'alice'}, \left\{ \begin{array}{l} \text{'lakers', 1} \\ \text{'iPod', 2} \end{array} \right\}, [\text{'age'} \rightarrow 20] \right)$$

Data Model

- “ Atom
- “ Tuple
- “ Bag - collection of tuples
 - “ Duplicates possible
 - “ Tuples in a bag can have different field lengths and field types
- “ Map

$$\left(\text{'alice'}, \left\{ \begin{array}{l} (\text{'lakers'}, 1) \\ (\text{'iPod'}, 2) \end{array} \right\}, [\text{'age'} \rightarrow 20] \right)$$

Data Model

- “ Atom
- “ Tuple
- “ Bag
- “ Map - collection of key-value pairs
 - “ Key is an atom; value can be any type

$$\left(\text{'alice'}, \left\{ \begin{array}{l} \text{'lakers', 1} \\ \text{'iPod', 2} \end{array} \right\}, [\text{'age'} \rightarrow 20] \right)$$

Data Model

“ Control over dataflow

Ex 1 (less efficient)

```
spam_urls = FILTER urls BY isSpam(url);  
culprit_urls = FILTER spam_urls BY pagerank > 0.8;
```

Ex 2 (most efficient)

```
highpgr_urls = FILTER urls BY pagerank > 0.8;  
spam_urls = FILTER highpgr_urls BY isSpam(url);
```

“ Fully nested

- “ More natural for procedural programmers (target user) than normalization

- “ Data is often stored on disk in a nested fashion

- “ Facilitates ease of writing user-defined functions

- “ No schema required

Data Model

“ User-Defined Functions (UDFs)

- “ Ex: `spam_urls = FILTER urls BY isSpam(url);`
- “ Can be used in many Pig Latin statements
- “ Useful for custom processing tasks
- “ Can use non-atomic values for input and output
- “ Currently must be written in Java

Speaking Pig Latin

“ LOAD

- “ Input is assumed to be a bag (sequence of tuples)
- “ Can specify a deserializer with “USING”
- “ Can provide a schema with “AS”

```
newBag = LOAD 'filename'  
        <USING functionName() >  
        <AS (fieldName1, fieldName2,...) >;
```

```
Queries = LOAD 'query_log.txt'  
        USING myLoad()  
        AS (userID, queryString, timeStamp)
```

Speaking Pig Latin

“ FOREACH

- “ Apply some processing to each tuple in a bag
- “ Each field can be:
 - “ A fieldname of the bag
 - “ A constant
 - “ A simple expression (ie: $f1 + f2$)
 - “ A predefined function (ie: SUM, AVG, COUNT, FLATTEN)
 - “ A UDF (ie: `sumTaxes(gst, pst)`)

```
newBag =  
    FOREACH bagName  
    GENERATE field1, field2, ...;
```

Speaking Pig Latin

“ FILTER

“ Select a subset of the tuples in a bag

```
newBag = FILTER bagName  
          BY expression;
```

“ Expression uses simple comparison operators (==, !=, <, >, ...) and Logical connectors (AND, NOT, OR)

```
some_apples =  
  FILTER apples BY colour != 'red';
```

“ Can use UDFs

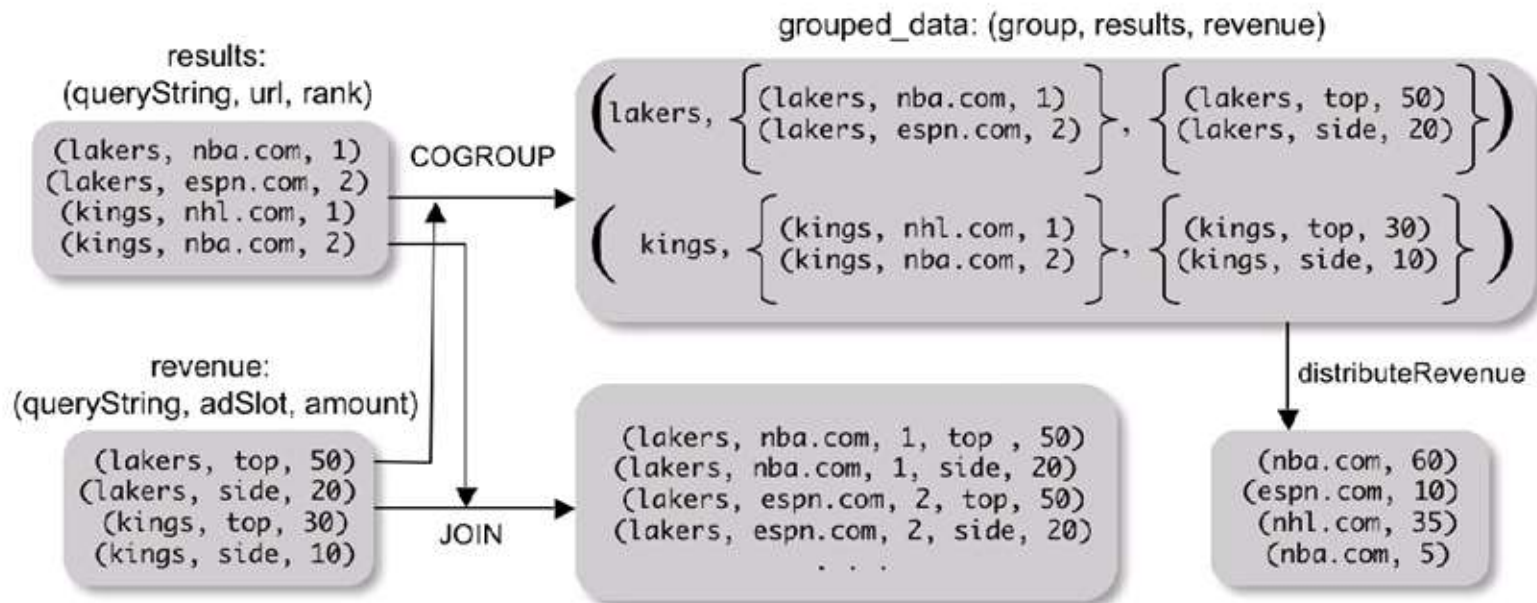
```
some_apples =  
  FILTER apples BY NOT isRed(colour);
```

Speaking Pig Latin

“ COGROUP

- “ Group two datasets together by a common attribute
- “ Groups data into nested bags

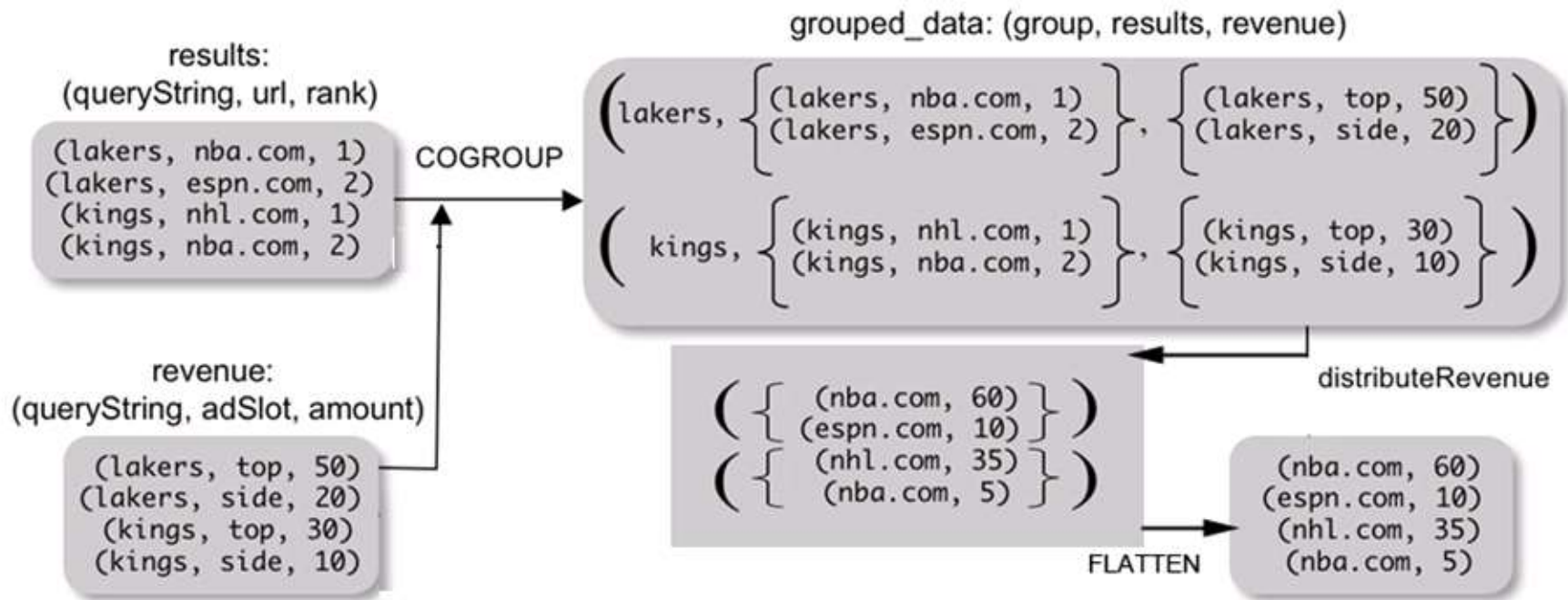
```
grouped_data = COGROUP results BY queryString,  
                        revenue BY queryString;
```



Speaking Pig Latin

“Why COGROUP and not JOIN?”

```
url_revenues =  
    FOREACH grouped_data GENERATE  
        FLATTEN(distributeRev(results, revenue));
```



Speaking Pig Latin

“Why COGROUP and not JOIN?

- “ May want to process nested bags of tuples before taking the cross product.
- “ Keeps to the goal of a single high-level data transformation per pig-latin statement.
- “ However, JOIN keyword is still available:

```
JOIN results BY queryString,  
    revenue BY queryString;
```



Equivalent

```
temp = COGROUP results BY queryString,  
    revenue BY queryString;  
join_result = FOREACH temp GENERATE  
    FLATTEN(results), FLATTEN(revenue);
```

Speaking Pig Latin

“ STORE (& DUMP)

“ Output data to a file (or screen)

```
STORE bagName INTO 'filename'  
    <USING deserializer ()>;
```

“ Other Commands (incomplete)

“ UNION - return the union of two or more bags

“ CROSS - take the cross product of two or more bags

“ ORDER - order tuples by a specified field(s)

“ DISTINCT - eliminate duplicate tuples in a bag

“ LIMIT - Limit results to a subset

Compilation

- “ Pig system does two tasks:
 - “ Builds a Logical Plan from a Pig Latin script
 - “ Supports execution platform independence
 - “ No processing of data performed at this stage
 - “ Compiles the Logical Plan to a Physical Plan and Executes
 - “ Convert the Logical Plan into a series of Map-Reduce statements to be executed (in this case) by Hadoop Map-Reduce

Compilation

“ Building a Logical Plan

- “ Verify input files and bags referred to are valid
- “ Create a logical plan for each bag(variable) defined

Compilation

“ Building a Logical Plan Example

```
A = LOAD 'user.dat' AS (name, age, city);  
B = GROUP A BY city;  
C = FOREACH B GENERATE group AS city,  
    COUNT(A);  
D = FILTER C BY city IS 'kitchener'  
    OR city IS 'waterloo';  
STORE D INTO 'local_user_count.dat';
```

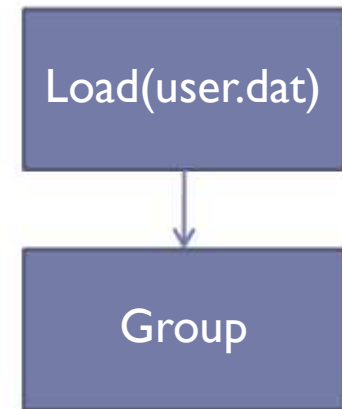


Load(user.dat)

Compilation

“ Building a Logical Plan Example

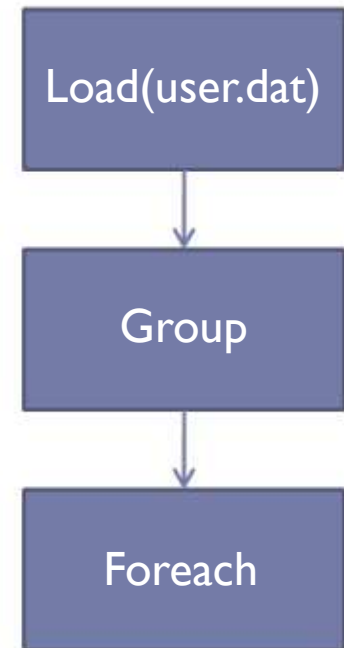
```
A = LOAD 'user.dat' AS (name, age, city);  
B = GROUP A BY city;  
C = FOREACH B GENERATE group AS city,  
    COUNT(A);  
D = FILTER C BY city IS 'kitchener'  
    OR city IS 'waterloo';  
STORE D INTO 'local_user_count.dat';
```



Compilation

“ Building a Logical Plan Example

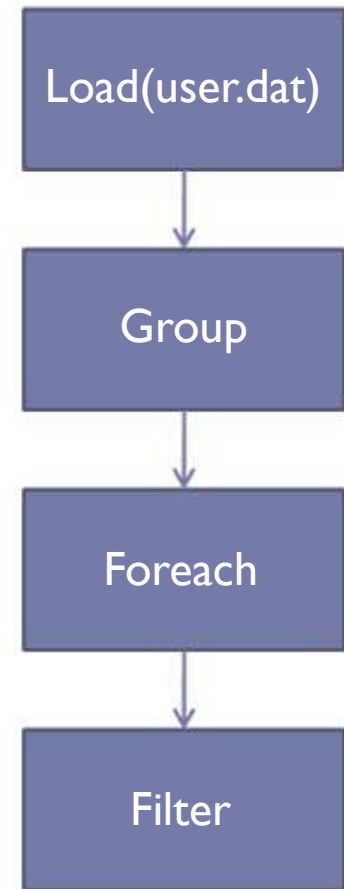
```
A = LOAD 'user.dat' AS (name, age, city);  
B = GROUP A BY city;  
C = FOREACH B GENERATE group AS city,  
    COUNT(A);  
D = FILTER C BY city IS 'kitchener'  
    OR city IS 'waterloo';  
STORE D INTO 'local_user_count.dat';
```



Compilation

“ Building a Logical Plan Example

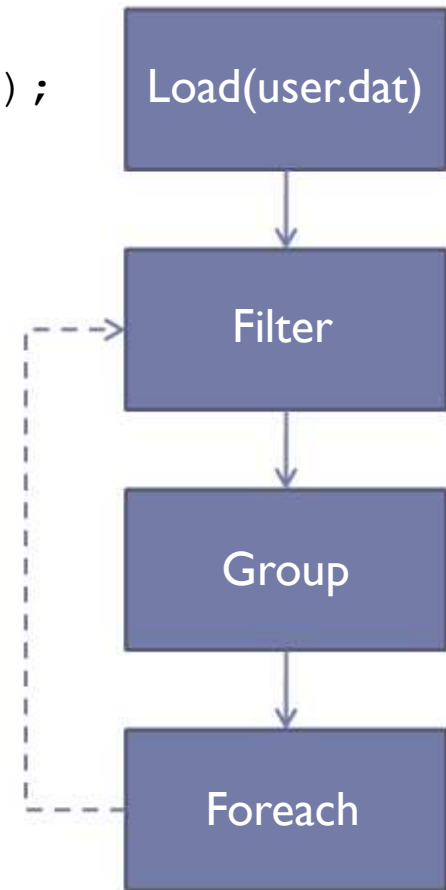
```
A = LOAD 'user.dat' AS (name, age, city);  
B = GROUP A BY city;  
C = FOREACH B GENERATE group AS city,  
    COUNT(A);  
D = FILTER C BY city IS 'kitchener'  
    OR city IS 'waterloo';  
STORE D INTO 'local_user_count.dat';
```



Compilation

“ Building a Logical Plan Example

```
A = LOAD 'user.dat' AS (name, age, city);  
B = GROUP A BY city;  
C = FOREACH B GENERATE group AS city,  
    COUNT(A);  
D = FILTER C BY city IS 'kitchener'  
    OR city IS 'waterloo';  
STORE D INTO 'local_user_count.dat';
```

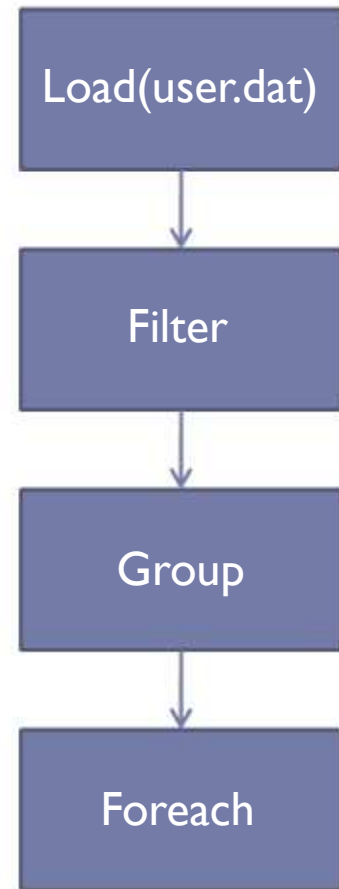


Compilation

“ Building a Physical Plan

```
A = LOAD 'user.dat' AS (name, age, city);  
B = GROUP A BY city;  
C = FOREACH B GENERATE group AS city,  
    COUNT(A);  
D = FILTER C BY city IS 'kitchener'  
    OR city IS 'waterloo';  
STORE D INTO 'local_user_count.dat';
```

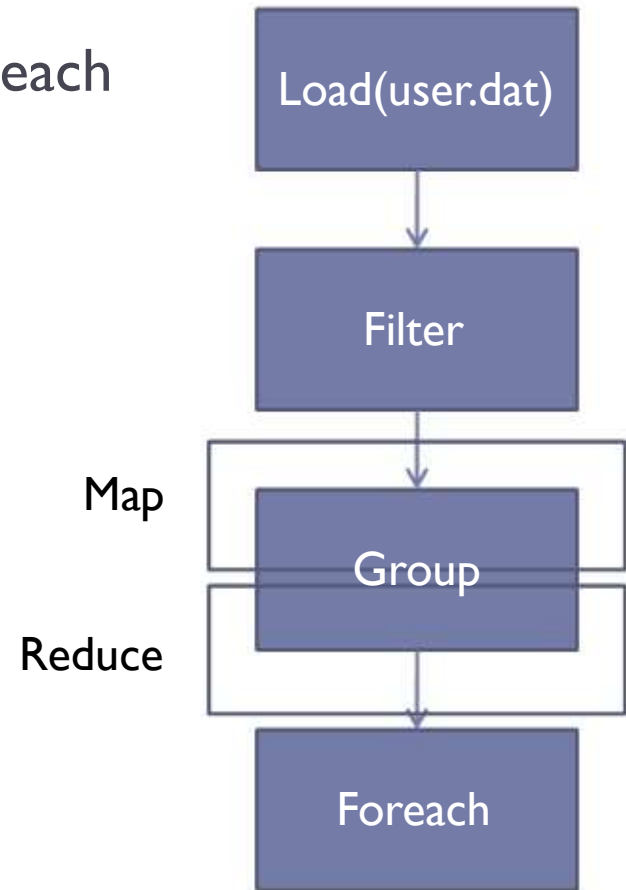
Only happens when output is
specified by STORE or DUMP



Compilation

“ Building a Physical Plan

“ Step 1: Create a map-reduce job for each COGROUP



Compilation

“ Building a Physical Plan

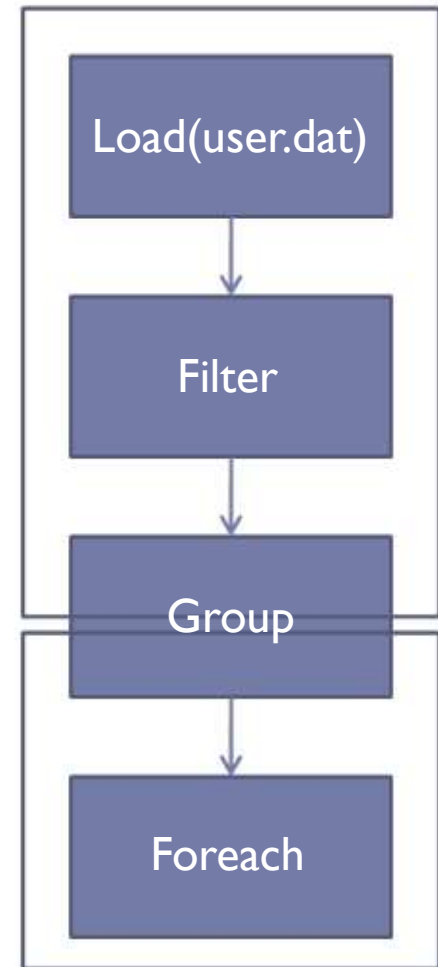
“ Step 1: Create a map-reduce job for each COGROUP

“ Step 2: Push other commands into the map and reduce functions where possible

“ May be the case certain commands require their own map-reduce job (ie: ORDER needs separate map-reduce jobs)

Map

Reduce



Compilation

- “ Efficiency in Execution

- “ Parallelism

- “ Loading data - Files are loaded from HDFS

- “ Statements are compiled into map-reduce jobs

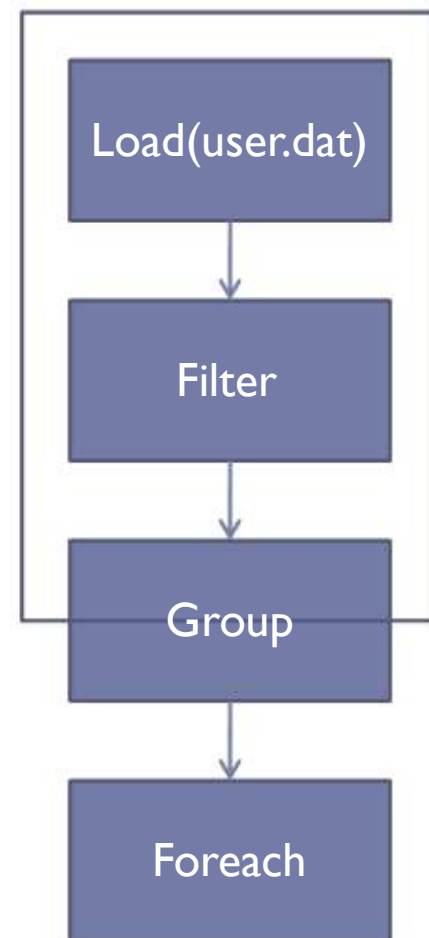
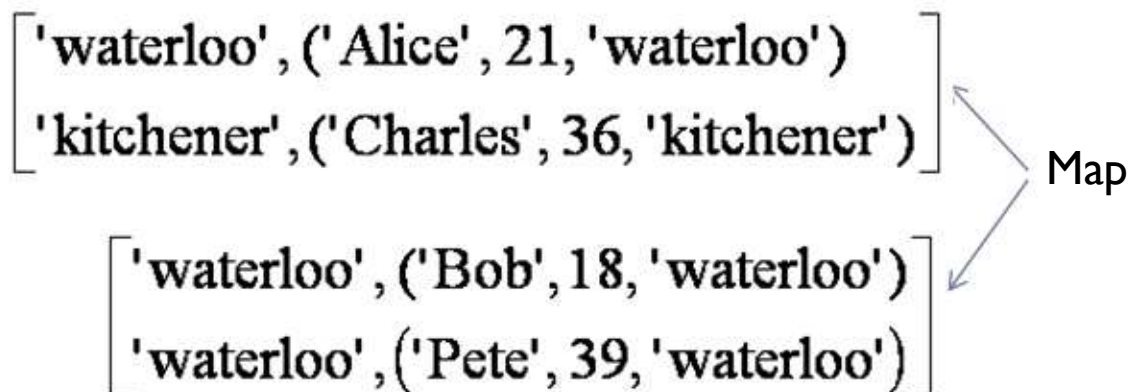
Compilation

“ Efficiency with Nested Bags

- “ In many cases, the nested bags created in each tuple of a COGROUP statement never need to physically materialize
- “ Generally perform aggregation after a COGROUP and the statements for said aggregation are pushed into the reduce function
- “ Applies to algebraic functions (ie: COUNT, MAX, MIN, SUM, AVG)

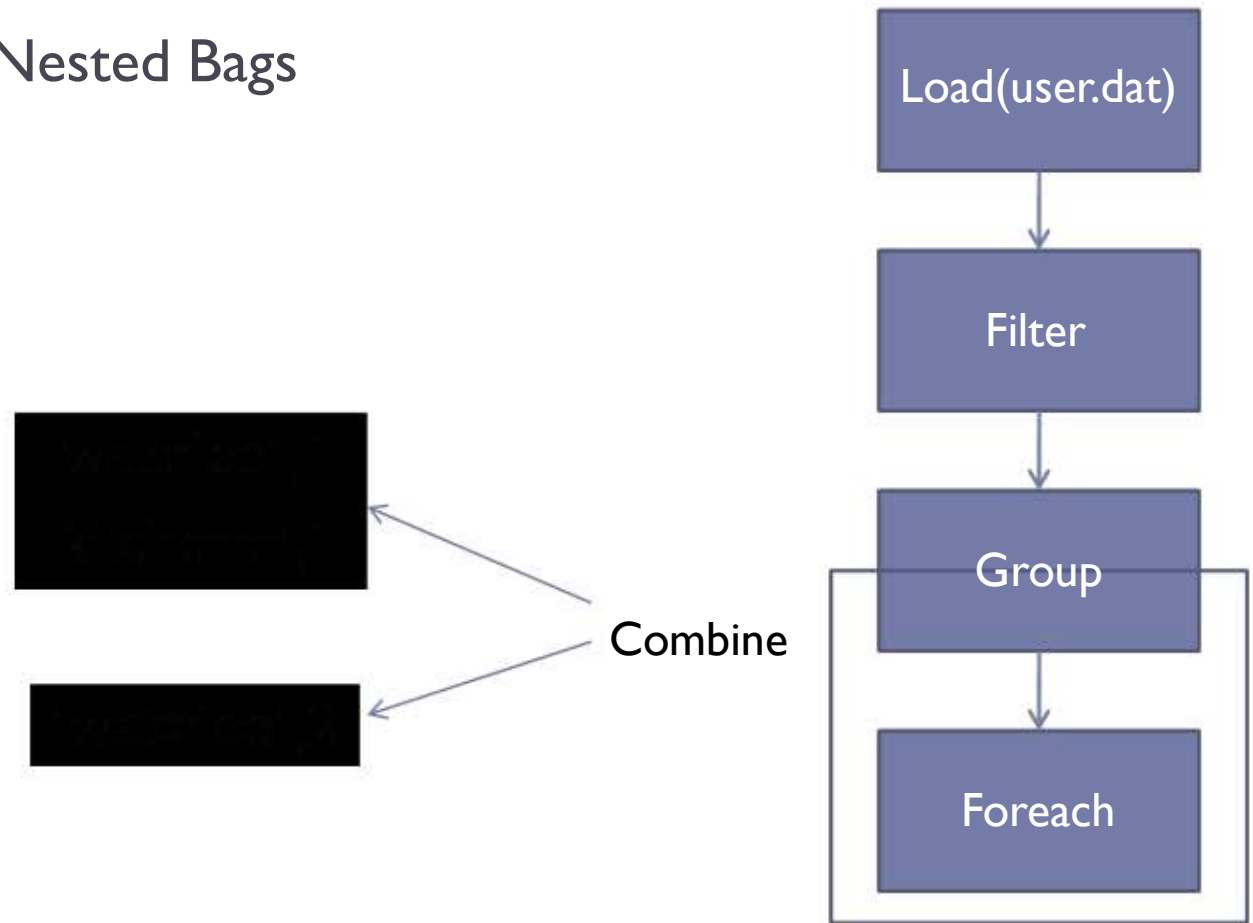
Compilation

“ Efficiency with Nested Bags



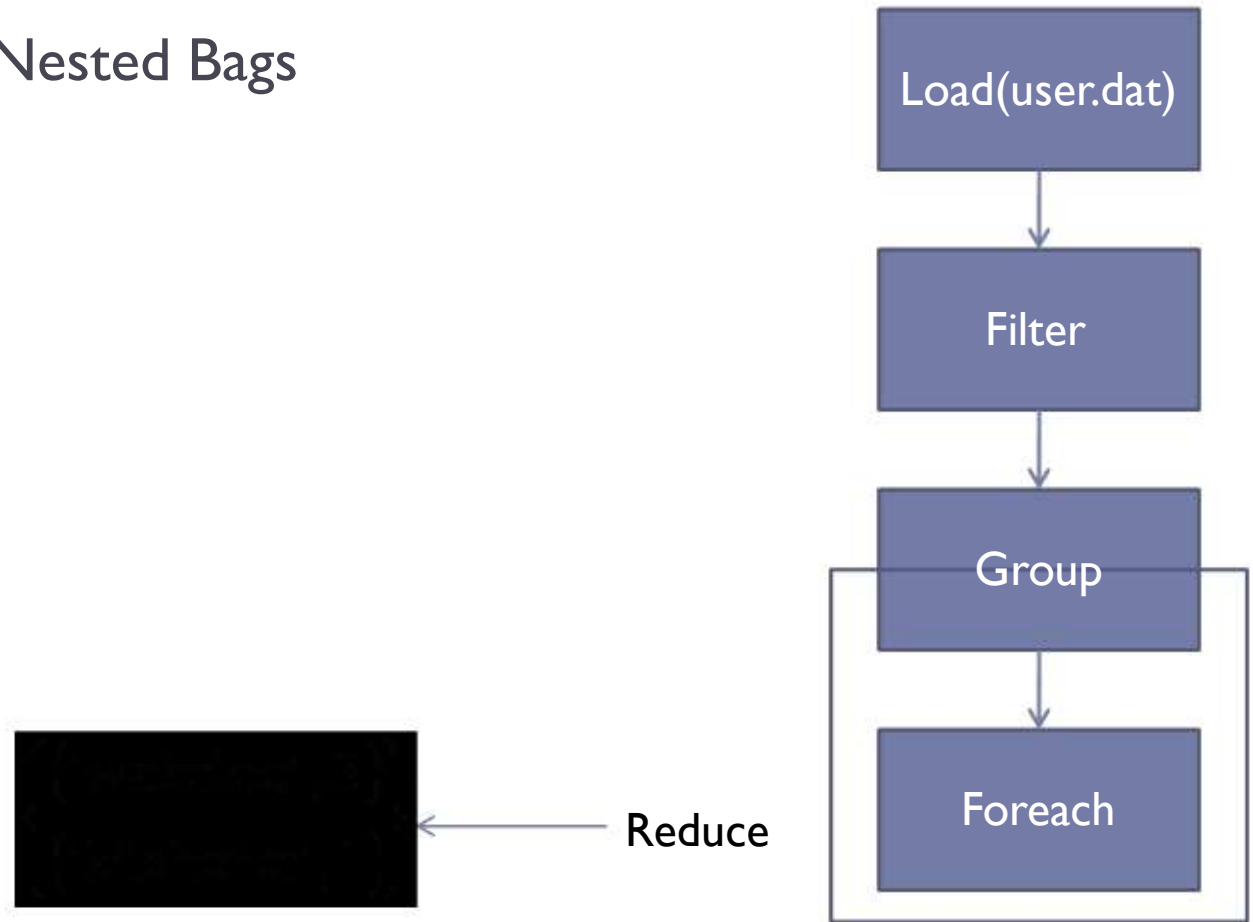
Compilation

“ Efficiency with Nested Bags



Compilation

“ Efficiency with Nested Bags

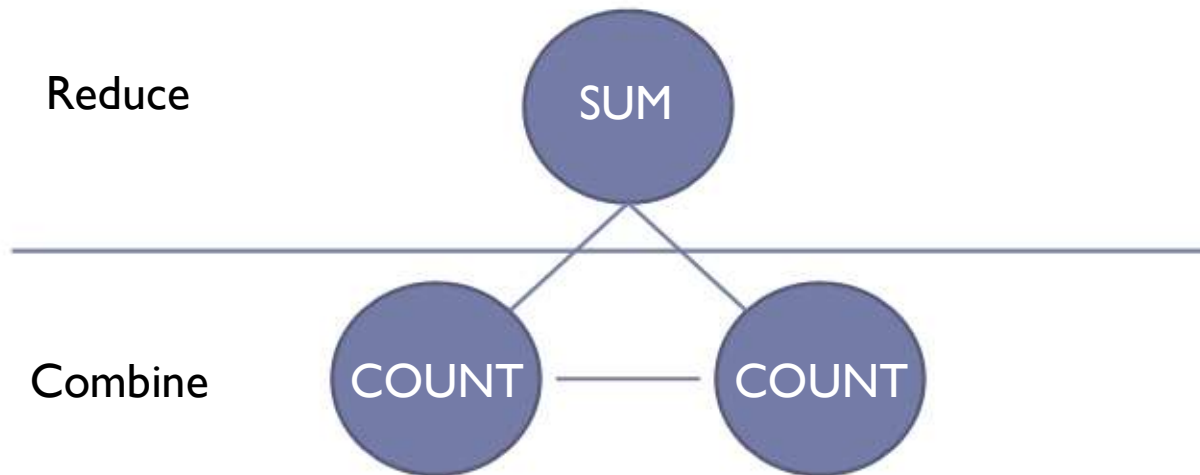


Compilation

“ Efficiency with Nested Bags

“ Why this works:

- COUNT is an algebraic function; it can be structured as a tree of sub-functions with each leaf working on a subset of the data



Compilation

“ Efficiency with Nested Bags

“ Pig provides an interface for writing algebraic UDFs so they can take advantage of this optimization as well.

“ Inefficiencies

“ Non-algebraic aggregate functions (ie: MEDIAN) need entire bag to materialize; may cause a very large bag to spill to disk if it doesn't fit in memory

“ Every map-reduce job requires data be written and replicated to the HDFS (although this is offset by parallelism achieved)

High-Level Language

```
raw = LOAD 'excite.log' USING PigStorage('\t') AS (user, id, time, query);
clean1 = FILTER raw BY id > 20 AND id < 100;
clean2 = FOREACH clean1 GENERATE
    user, time,
    org.apache.pig.tutorial.sanitize(query) as query;
user_groups = GROUP clean2 BY (user, query);
user_query_counts = FOREACH user_groups
    GENERATE group, COUNT(clean2), MIN(clean2.time), MAX(clean2.time);
STORE user_query_counts INTO 'uq_counts.csv' USING PigStorage(',');
```

Pig Components

Two Main Components

- **High-level language (Pig Latin)**
 - Set of commands
- **Two execution modes**
 - Local: reads/write to local file system
 - Mapreduce: connects to Hadoop cluster and reads/writes to HDFS

Two modes

- **Interactive mode**
 - Console
- **Batch mode**
 - Submit a script

Why Pig?...Abstraction!

- Common design patterns as key words (joins, distinct, counts)
- Data flow analysis
 - A script can map to multiple map-reduce jobs
- Avoids Java-level errors (not everyone can write java code)
- Can be interactive mode
 - Issue commands and get results

Example I: More Details

Read file from HDFS

The input format (text, tab delimited)

Define run-time schema

```
raw = LOAD 'excite.log' USING PigStorage('\t') AS (user, id, time, query);  
clean1 = FILTER raw BY id > 20 AND id < 100;  
clean2 = FOREACH clean1 GENERATE  
        user, time,  
        org.apache.pig.tutorial.sanitize(query) as query;  
user_groups = GROUP clean2 BY (user, query);  
user_query_counts = FOREACH user_groups  
    GENERATE group, COUNT(clean2), MIN(clean2.time), MAX(clean2.time);  
STORE user_query_counts INTO 'uq_counts.csv' USING PigStorage(',');
```

Filter the rows on predicates

For each row, do some transformation

Grouping of records

Compute aggregation for each group

Store the output in a file

Text, Comma delimited

Pig: Language Features

- **Keywords**
 - Load, Filter, Foreach Generate, Group By, Store, Join, Distinct, Order By, ...
- **Aggregations**
 - Count, Avg, Sum, Max, Min
- **Schema**
 - Defines at query-time not when files are loaded
- UDFs
- Packages for common input/output formats

Example 2

Script can take arguments

Data are “ctrl-A” delimited

Define types of the columns

A = **load** '\$widerow' **using** PigStorage('\u0001') **as** (name: chararray, c0: int, c1: int, c2: int);

B = **group** A **by** name **parallel** 10; ← Specify the need of 10 reduce tasks

C = **foreach** B **generate** group, SUM(A.c0) as c0, SUM(A.c1) as c1, AVG(A.c2) as c2;

D = **filter** C **by** c0 > 100 and c1 > 100 and c2 > 100;

store D **into** '\$out';

Example 3: Re-partition Join

Register UDFs & custom inputformats

register pigperf.jar;

Function the jar file to read the input file

A = **load** 'page_views' **using** org.apache.pig.test.udf.storefunc.PigPerformanceLoader()
 as (user, action, timespent, query_term, timestamp, estimated_revenue);

B = **foreach** A **generate** user, (double) estimated_revenue;

Load the second file

alpha = **load** 'users' **using** PigStorage('\u0001') **as** (name, phone, address, city, state, zip);

beta = **foreach** alpha **generate** name, city;

Join the two datasets (40 reducers)

C = **join** beta **by** name, B **by** user **parallel** 40;

D = **group** C **by** \$0; ← Group after the join (can reference columns by position)

E = **foreach** D **generate** group, SUM(C.estimated_revenue);

store E **into** 'L3out';

This grouping can be done in the same map-reduce job because it is on the same key (Pig can do this optimization)

Example 4: Replicated Join

```
register pigperf.jar;
```

```
A = load 'page_views' using org.apache.pig.test.udf.storefunc.PigPerformanceLoader()  
    as (user, action, timespent, query_term, timestamp, estimated_revenue);
```

```
Big = foreach A generate user, (double) estimated_revenue;
```

```
alpha = load 'users' using PigStorage('\u0001') as (name, phone, address, city, state, zip);
```

```
small = foreach alpha generate name, city;
```

```
C = join Big by user, small by name using 'replicated';
```

```
store C into 'out';
```

Map-only join (the small dataset is the second)



Optimization in joining a big dataset with a small one

Example 5: Multiple Outputs

```
A = LOAD 'data' AS (f1:int,f2:int,f3:int);
```

```
DUMP A;
```

```
(1,2,3)
```

```
(4,5,6)
```

```
(7,8,9)
```

Split the records into sets

```
SPLIT A INTO X IF f1<7, Y IF f2==5, Z IF (f3<6 OR f3>6);
```

```
DUMP X;
```

```
(1,2,3)
```

```
(4,5,6)
```

Dump command to display the data

```
DUMP Y;
```

```
(4,5,6)
```

Store multiple outputs

```
STORE x INTO 'x_out';
```

```
STORE y INTO 'y_out';
```

```
STORE z INTO 'z_out';
```

Run independent jobs in parallel

```
D1 = load 'data1' ...
```

```
D2 = load 'data2' ...
```

```
D3 = load 'data3' ...
```

```
C1 = join D1 by a, D2 by b
```

```
C2 = join D1 by c, D3 by d
```

C1 and C2 are two independent jobs that can run in parallel

Pig Latin vs. SQL

- **In Pig Latin**

- Lazy evaluation (data not processed prior to STORE command)
- Data can be stored at any point during the pipeline
- Schema and data types are lazily defined at run-time
- An execution plan can be explicitly defined
 - Use optimizer hints
 - Due to the lack of complex optimizers

- **In SQL:**

- Query plans are solely decided by the system
- Data cannot be stored in the middle
- Schema and data types are defined at the creation time

Pig References

- **Pig Tutorial**
 - <http://pig.apache.org/docs/r0.7.0/tutorial.html>
- **Pig Latin Reference Manual 2**
 - http://pig.apache.org/docs/r0.7.0/piglatin_ref1.html
- **Pig Latin Reference Manual 2**
 - http://pig.apache.org/docs/r0.7.0/piglatin_ref2.html
- **PigMix Queries**
 - <https://cwiki.apache.org/PIG/pigmix.html>

Apache Pig



Hive

Hive

The Hadoop ecosystem contains different sub-projects (tools) such as Sqoop, Pig, and Hive that are used to help Hadoop modules.

•**Sqoop:** It is used to import and export data to and from between HDFS and RDBMS.

•**Pig:** It is a procedural language platform used to develop a script for MapReduce operations.

•**Hive:** It is a platform used to develop SQL type scripts to do MapReduce operations.

Note: There are various ways to execute MapReduce operations:

- The traditional approach using Java MapReduce program for structured, semi-structured, and unstructured data.
- The scripting approach for MapReduce to process structured and semi structured data using Pig.
- The Hive Query Language (HiveQL or HQL) for MapReduce to process structured data using Hive.

What is Hive

Hive is a data warehouse infrastructure tool to process structured data in Hadoop. It resides on top of Hadoop to summarize Big Data, and makes querying and analyzing easy.

Initially Hive was developed by Facebook, later the Apache Software Foundation took it up and developed it further as an open source under the name Apache Hive. It is used by different companies. For example, Amazon uses it in Amazon Elastic MapReduce.

Hive is not

- A relational database
- A design for OnLine Transaction Processing (OLTP)
- A language for real-time queries and row-level updates

Features of Hive

- It stores schema in a database and processed data into HDFS.
- It is designed for OLAP.
- It provides SQL type language for querying called HiveQL or HQL.
- It is familiar, fast, scalable, and extensible.

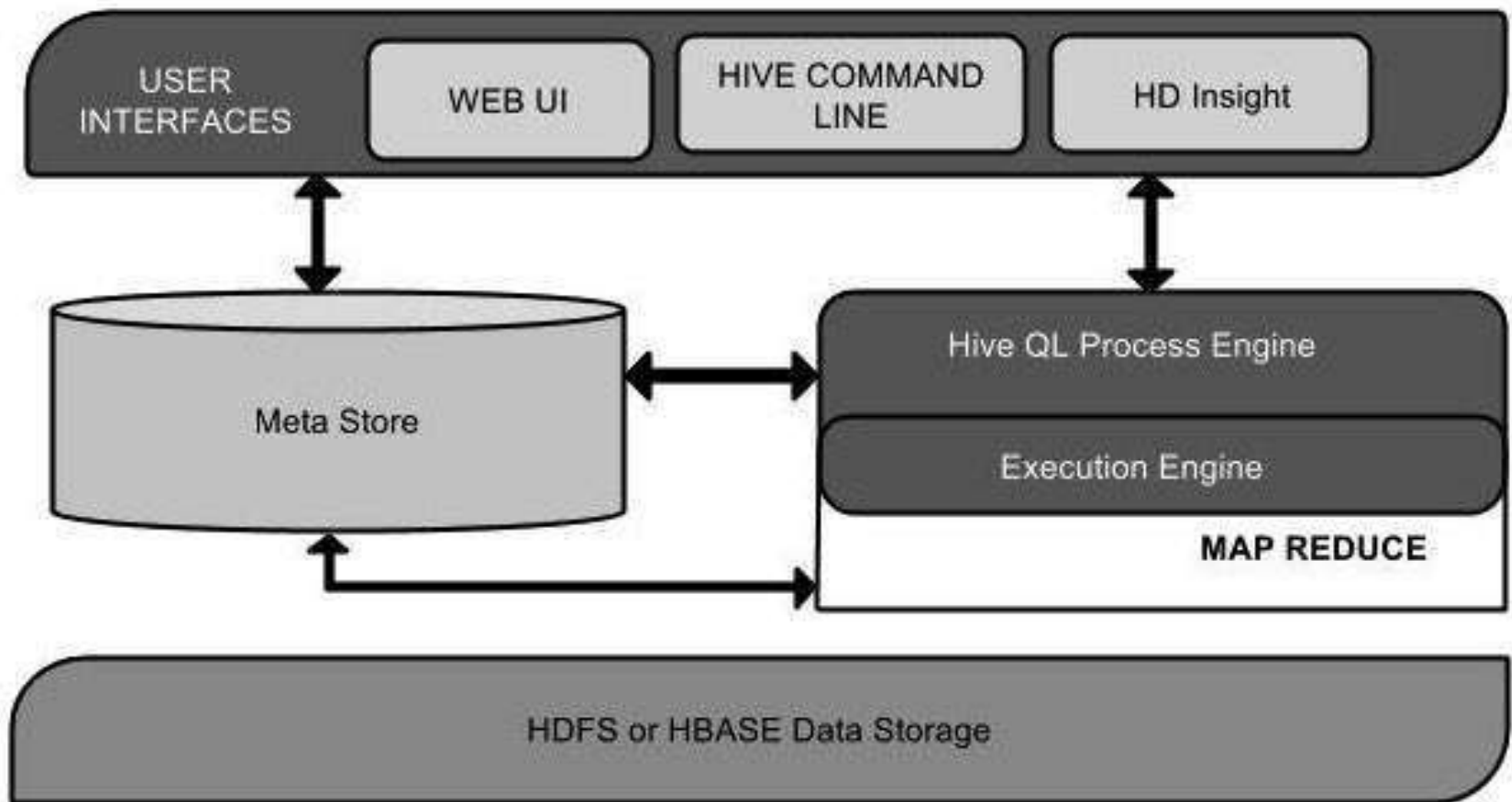
Apache Hive

- A data warehouse infrastructure built on top of Hadoop for providing data summarization, query, and analysis
- **Hive Provides**
 - ETL
 - Structure
 - Access to different storage (HDFS or HBase)
 - Query execution via MapReduce
- **Key Building Principles**
 - SQL is a familiar language
 - Extensibility – Types, Functions, Formats, Scripts
 - Performance



Architecture of Hive

The following component diagram depicts the architecture of Hive:



This component diagram contains different units.
The following table describes each unit:

Unit Name	Operation
User Interface	Hive is a data warehouse infrastructure software that can create interaction between user and HDFS. The user interfaces that Hive supports are Hive Web UI, Hive command line, and Hive HD Insight (In Windows server).
Meta Store	Hive chooses respective database servers to store the schema or Metadata of tables, databases, columns in a table, their data types, and HDFS mapping.
HiveQL Process Engine	HiveQL is similar to SQL for querying on schema info on the Metastore. It is one of the replacements of traditional approach for MapReduce program. Instead of writing MapReduce program in Java, we can write a query for MapReduce job and process it.
Execution Engine	The conjunction part of HiveQL process Engine and MapReduce is Hive Execution Engine. Execution engine processes the query and generates results as same as MapReduce results. It uses the flavor of MapReduce.
HDFS or HBASE	Hadoop distributed file system or HBASE are the data storage techniques to store data into file system.

Hive Components

Two Main Components

- **High-level language (HiveQL)**
 - Set of commands
- **Two execution modes**
 - Local: reads/write to local file system
 - Mapreduce: connects to Hadoop cluster and reads/writes to HDFS

Two modes

- **Interactive mode**
 - Console
- **Batch mode**
 - Submit a script

Hive deals with Structured Data

- Data Units
 - Databases
 - Tables
 - Partitions
 - Buckets (or clusters)

Very similar to SQL and Relational DBs

Hive DDL Commands

- ▶ **CREATE TABLE** sample (foo INT, bar STRING) **PARTITIONED BY** (ds STRING);
- ▶ **SHOW TABLES** '.*s';
- ▶ **DESCRIBE** sample;
- ▶ **ALTER TABLE** sample **ADD COLUMNS** (new_col INT);
- ▶ **DROP TABLE** sample;

```
create table table_name (  
  id          int,  
  dtDontQuery string,  
  name        string  
)  
partitioned by (date string)
```

A table in Hive is an HDFS
directory in Hadoop

Schema is known at creation time (like DB schema)

Partitioned tables have “sub-directories”, one for each partition

Hive DML

Load data from local file system

Delete previous data from that table

▶ **LOAD DATA LOCAL INPATH** './sample.txt' **OVERWRITE INTO TABLE** sample;

Load data from HDFS

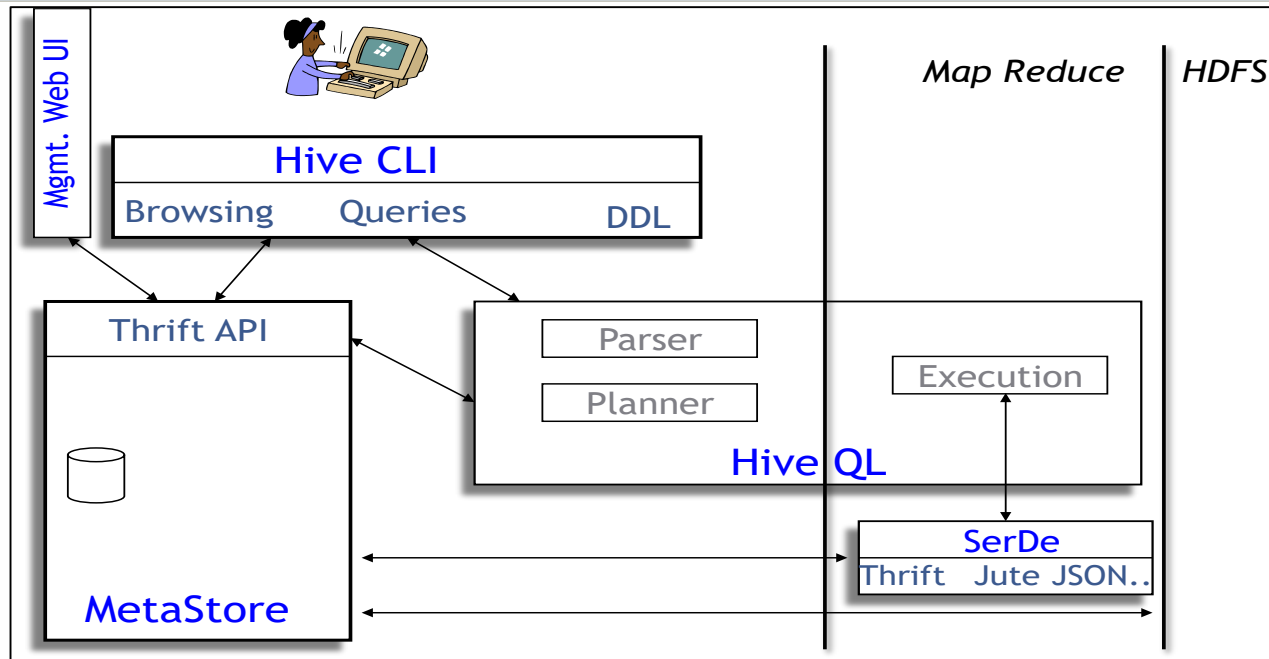
Augment to the existing data

▶ **LOAD DATA INPATH** '/user/falvariz/hive/sample.txt' **INTO TABLE** partitioned_sample **PARTITION** (ds='2012-02-24');

Must define a specific partition for partitioned tables

Loaded data are files copied to HDFS under the corresponding directory

Hive Components

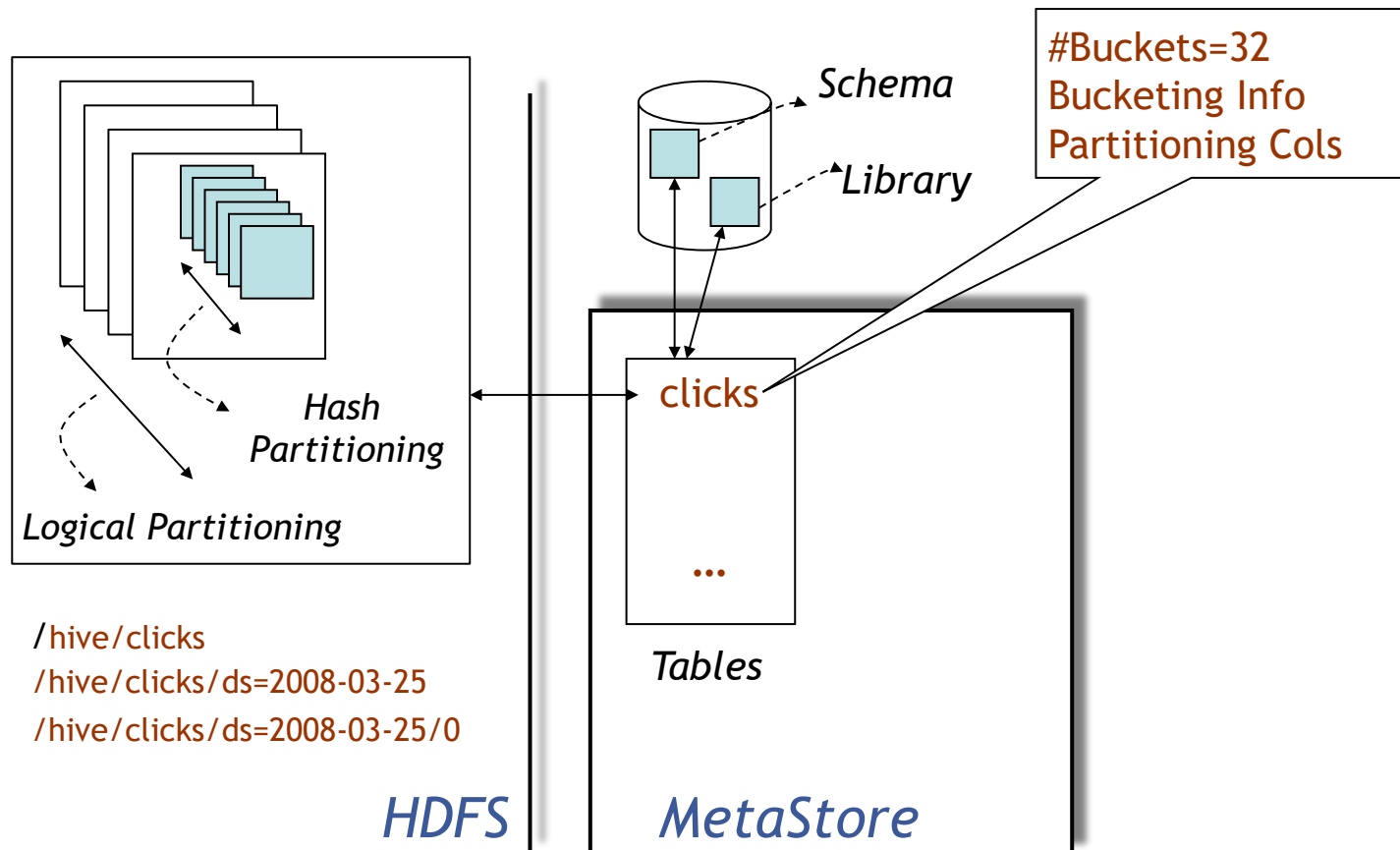


- Hive CLI: Hive Client Interface
- MetaStore: For storing the schema information, data types, partitioning columns, etc...
- Hive QL: The query language, compiler, and executer

Data Model

- **3-Levels:** Tables → Partitions → Buckets
- **Table: maps to a HDFS directory**
 - Table R: Users all over the world
- **Partition: maps to sub-directories under the table**
 - Partition R by country name
 - It is the user's responsibility to upload the right data to the right partition
- **Bucket: maps to files under each partition**
 - Divide a partition into buckets based on a hash function on a certain column(s)

Data Model (Cont'd)



Query Examples I: Select & Filter

▶ **SELECT** foo **FROM** sample **WHERE** ds='2012-02-24';

Create HDFS dir for the output

▶ **INSERT OVERWRITE DIRECTORY** '/tmp/hdfs_out' **SELECT** *
FROM sample **WHERE** ds='2012-02-24';

Create local dir for the output

▶ **INSERT OVERWRITE LOCAL DIRECTORY** '/tmp/hive-sample-out' **SELECT** * **FROM** sample;

Query Examples II: Aggregation & Grouping

▶ **SELECT** MAX(foo) **FROM** sample;

▶ **SELECT** ds, COUNT(*), SUM(foo) **FROM** sample **GROUP BY** ds;

Hive allows the From clause to come first !!!

Store the results into a table

▶ **FROM** sample s **INSERT OVERWRITE TABLE** bar **SELECT** s.bar, count(*)
WHERE s.foo > 0 **GROUP BY** s.bar;

This new syntax is to facilitate the “Multi-Insertion”

Query Examples III: Multi-Insertion

```
FROM page_view_stg pvs
```

```
INSERT OVERWRITE TABLE page_view PARTITION(dt='2008-06-08', country='US')
```

```
SELECT pvs.viewTime, ... WHERE pvs.country = 'US'
```

```
INSERT OVERWRITE TABLE page_view PARTITION(dt='2008-06-08', country='CA')
```

```
SELECT pvs.viewTime, ... WHERE pvs.country = 'CA'
```

```
INSERT OVERWRITE TABLE page_view PARTITION(dt='2008-06-08', country='UK')
```

```
SELECT pvs.viewTime, ... WHERE pvs.country = 'UK';
```


Example IV: Joins

```
CREATE TABLE customer (id INT,name STRING,address STRING)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY '#';
```

```
CREATE TABLE order_cust (id INT,cus_id INT,prod_id INT,price INT)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
```

```
▶ SELECT * FROM customer c JOIN order_cust o ON (c.id=o.cus_id);  
  
▶ SELECT c.id, c.name, c.address, ce.exp  
FROM customer c JOIN (SELECT cus_id,sum(price) AS exp  
                        FROM order_cust  
                        GROUP BY cus_id) ce ON (c.id=ce.cus_id);
```